Modul 2 — DML (Data Manipulation Language)

Mata pelajaran: Basis Data — RPL **Alat:** MySQL/MariaDB (XAMPP), phpMyAdmin atau terminal **Studi kasus:** Pengelolaan data sekolah (`db_sekolah`) **Prasyarat:** Modul 1 (DDL) selesai; keempat tabel `jurusan`, `siswa`, `mapel`, `nilai` sudah ada.

---

1. Tujuan Pembelajaran

Setelah mempelajari modul ini, kamu mampu:

1. Menjelaskan fungsi DML dan perbedaannya dengan DDL.
2. Menambahkan data dengan `INSERT` (satu baris, banyak baris, dan dari hasil query).
3. Mengubah data dengan `UPDATE` secara aman.
4. Menghapus data dengan `DELETE` secara aman.
5. Memahami dampak foreign key (`CASCADE`, `SET NULL`) saat data diubah atau dihapus.

---

2. Konsep Dasar

DML (Data Manipulation Language) adalah perintah SQL untuk **mengelola isi tabel**: menambah, mengubah, dan menghapus baris data.

Perintah	Fungsi	Analogi
`INSERT`	Menambah baris baru	Menulis catatan baru di buku
`UPDATE`	Mengubah baris yang ada	Mengoreksi catatan
`DELETE`	Menghapus baris	Mencoret/membuang catatan

Perbedaan DDL dan DML

	DDL	DML
Objek	Struktur (database, tabel, kolom)	Isi (baris data)
Contoh	`CREATE TABLE siswa`	`INSERT INTO siswa ...`

---

3. Studi Kasus

Awal tahun ajaran, operator sekolah harus:

1. Memasukkan data jurusan, siswa, mapel, dan nilai awal.
2. Memperbaiki data yang salah ketik atau berubah (nama, email, jurusan, nilai).
3. Menghapus data yang tidak berlaku.

Semua pekerjaan itu dilakukan dengan DML.

---

4. Persiapan

```
USE db_sekolah; -- pilih database yang akan dikelola
SHOW TABLES;    -- pastikan tabel yang dibutuhkan sudah tersedia
```

Pastikan keempat tabel tampil. `USE` tidak membuat database; perintah ini hanya memilih database aktif. Jalankan modul ini **berurutan** pada database latihan, bukan database proyek yang berisi data penting.

---

5. Materi dan Praktik

5.1 `INSERT INTO` — menambah data

Sintaks dasar

```
INSERT INTO nama_tabel (kolom1, kolom2, ...) -- sebutkan tabel dan kolom yang akan diisi
VALUES (nilai1, nilai2, ...);                -- isi harus berurutan sesuai daftar kolom
```

Nama kolom dan nilai harus memiliki jumlah serta urutan yang sesuai. Kolom yang tidak dicantumkan akan memakai nilai `DEFAULT` atau `NULL` jika diizinkan.

a. Mengisi data awal (jalankan satu kali)

Urutan pengisian mengikuti relasi: **tabel induk dulu**, baru tabel anak.

```
-- 1. Tabel induk
INSERT INTO jurusan (nama_jurusan) VALUES
('Rekayasa Perangkat Lunak'),       -- baris pertama
('Desain Komunikasi Visual'),       -- baris kedua
('Teknik Komputer dan Jaringan'),   -- baris ketiga
('Tata Boga');                      -- baris keempat; titik koma mengakhiri perintah

INSERT INTO mapel (nama_mapel) VALUES
('Pemrograman Web'),
('Basis Data'),
('Pemrograman Berorientasi Objek'),
('Algoritma Pemrograman');

-- 2. Tabel anak
INSERT INTO siswa
    (nama, email, jenis_kelamin, tanggal_lahir, id_jurusan) -- kolom tujuan, berurutan
VALUES
    ('Andi', 'andi@example.com', 'L', '2008-03-12', 1), -- setiap baris mengikuti urutan kolom di atas
    ('Siti', 'siti@example.com', 'P', '2008-07-21', 1),
    ('Budi', 'budi@example.com', 'L', '2007-11-05', 2),
    ('Rina', 'rina@example.com', 'P', '2008-01-15', 3),
    ('Dewi', 'dewi@example.com', 'P', '2008-09-09', NULL); -- NULL: jurusan belum ditentukan

INSERT INTO nilai (id_siswa, id_mapel, nilai_angka, semester) VALUES
    (1, 1, 90, 1), -- siswa ID 1 mendapat nilai 90 untuk mapel ID 1 pada semester 1
    (1, 2, 85, 1),
    (1, 3, 88, 1),
    (2, 1, 95, 1),
    (2, 2, 91, 1),
    (3, 1, 78, 1),
    (3, 2, 82, 1),
    (4, 1, 87, 1);
```

Penjelasan singkat:

- `id_*` tidak diisi karena `AUTO_INCREMENT`.
- Teks dan tanggal diapit tanda petik tunggal; tanggal berformat `YYYY-MM-DD`.
- Dewi sengaja diberi `id_jurusan = NULL` (belum punya jurusan). Data ini dipakai di Modul 3 untuk latihan `JOIN`.
- Banyak baris sekaligus dalam satu `INSERT` lebih efisien.
- Angka `id_siswa` dan `id_mapel` pada tabel `nilai` mengacu pada ID yang dibuat otomatis. Angka contoh ini sesuai untuk database latihan yang baru diisi; jika data pernah diubah, periksa ID sebenarnya terlebih dahulu.

Periksa hasil:

```
SELECT * FROM jurusan; -- tampilkan semua kolom dan baris jurusan
SELECT * FROM siswa;   -- periksa data siswa dan ID yang terbentuk
SELECT * FROM mapel;   -- periksa mata pelajaran dan ID-nya
SELECT * FROM nilai;   -- periksa hubungan siswa, mapel, nilai, dan semester
```

`*` berarti semua kolom. Hasil yang diharapkan pada data awal adalah 3 jurusan, 5 siswa, 3 mapel, dan 8 nilai.

> Jika muncul `Duplicate entry`, data kemungkinan sudah pernah dimasukkan. Periksa isi tabel dulu, jangan mengulang seluruh `INSERT`.

b. Menambah satu siswa

```
INSERT INTO siswa
    (nama, email, jenis_kelamin, tanggal_lahir, id_jurusan) -- kolom yang diisi
VALUES
    ('Fajar', 'fajar@example.com', 'L', '2008-05-20', 1); -- satu siswa; jurusan harus sudah ada
```

Penjelasan: menambah satu siswa pada jurusan dengan ID 1. ID siswa dibuat otomatis, tetapi angkanya belum tentu 6 jika tabel pernah diisi atau dihapus sebelumnya; periksa dengan `SELECT` setelah `INSERT`.

c. Mengisi sebagian kolom

```
INSERT INTO siswa (nama, jenis_kelamin)
VALUES ('Gita', 'P'); -- kolom lain memakai DEFAULT atau NULL jika diizinkan
```

Penjelasan per bagian: hanya `nama` dan `jenis_kelamin` yang diisi. `id_siswa` dibuat otomatis, sedangkan kolom lain yang tidak disebut akan memakai nilai `DEFAULT` atau `NULL` jika diizinkan. Jika kolom wajib (`NOT NULL`) tidak memiliki nilai default, MySQL akan menolak baris ini.

d. Error `INSERT` yang sering muncul

Query	Hasil	Sebab
Memasukkan email yang sudah ada	`Duplicate entry`	Constraint `UNIQUE`
`jenis_kelamin` diisi `'X'`	Error / ditolak	`ENUM('L','P')`
`id_jurusan` diisi `99`	Foreign key error	Jurusan 99 tidak ada
`nama` tidak diisi	Error	`NOT NULL`

Pelajaran: error adalah tanda constraint sedang menjaga konsistensi data.

5.2 `UPDATE` — mengubah data

Sintaks dasar

```
UPDATE nama_tabel       -- pilih tabel yang akan diubah
SET kolom = nilai_baru  -- tetapkan nilai baru
WHERE kondisi;          -- batasi baris yang cocok; tanpa WHERE semua baris terkena
```

> **Kebiasaan wajib:** jalankan `SELECT` dengan `WHERE` yang sama dulu untuk memastikan baris yang akan terdampak sudah benar. Tanpa `WHERE`, **semua baris** berubah.

a. Mengubah satu kolom

Kasus: nama Andi ternyata lengkapnya "Andi Pratama".

```
-- Periksa dulu
SELECT * FROM siswa WHERE id_siswa = 1;

-- Ubah
UPDATE siswa
SET nama = 'Andi Pratama' -- isi nama baru
WHERE id_siswa = 1;        -- ubah hanya baris dengan ID siswa 1
```

b. Mengubah beberapa kolom

Kasus: Andi pindah jurusan ke DKV dan mengganti email.

```
UPDATE siswa
SET email = 'andi.pratama@example.com',
    id_jurusan = 2         -- kedua kolom diubah dalam satu perintah
WHERE id_siswa = 1;        -- kondisi yang menentukan baris Andi
```

Penjelasan: pisahkan pasangan `kolom = nilai_baru` dengan koma dalam satu `SET`. `WHERE` memilih baris siswa yang akan diubah; email baru tetap harus unik dan ID jurusan harus ada.

c. Mengubah banyak baris dengan kondisi

Kasus: guru memberi tambahan 5 poin untuk semua nilai Pemrograman Web semester 1.

```
-- Periksa dulu
SELECT * FROM nilai WHERE id_mapel = 1 AND semester = 1;

UPDATE nilai
SET nilai_angka = nilai_angka + 5 -- hitung nilai baru dari nilai yang sedang tersimpan
WHERE id_mapel = 1                -- pilih catatan untuk mapel ID 1
  AND semester = 1;               -- sekaligus batasi ke semester 1
```

Penjelasan: `nilai_angka + 5` memakai nilai lama sebagai dasar.

> **Jangan dijalankan berulang**, nilai akan terus bertambah. Perhatikan juga batas `DECIMAL(5,2)` dan aturan nilai maksimal 100 pada kasus nyata (bisa dijaga dengan `LEAST(nilai_angka + 5, 100)`).

d. Contoh `UPDATE` berbahaya (jangan dijalankan)

```
-- Tanpa WHERE: SEMUA siswa berubah
-- UPDATE siswa SET nama = 'Andi';
```

5.3 `DELETE` — menghapus data

Sintaks dasar

```
DELETE FROM nama_tabel -- pilih tabel yang barisnya akan dihapus
WHERE kondisi;         -- batasi baris yang cocok; tanpa WHERE semua baris terhapus
```

a. Menghapus satu baris

Kasus: data Dewi (ID 5) dihapus karena pindah sekolah.

```
-- Periksa dulu
SELECT * FROM siswa WHERE id_siswa = 5;

DELETE FROM siswa
WHERE id_siswa = 5; -- hapus hanya siswa dengan ID 5
```

Penjelasan: karena `ON DELETE CASCADE` pada tabel `nilai`, nilai milik siswa tersebut (jika ada) ikut terhapus. Dewi belum punya nilai, sehingga tidak ada nilai yang hilang.

b. Menghapus satu catatan nilai

```
DELETE FROM nilai
WHERE id_nilai = 8; -- hapus hanya catatan nilai dengan ID 8
```

`DELETE FROM` menghapus baris, bukan struktur tabel. Sebelum menjalankan perintah, uji kondisi yang sama dengan `SELECT` dan pastikan baris yang tampil memang ingin dihapus.

c. Menghapus berdasarkan kondisi (banyak baris)

```
-- Periksa dulu: berapa siswa yang akan terhapus?
SELECT * FROM siswa WHERE id_jurusan = 3;

DELETE FROM siswa
WHERE id_jurusan = 3; -- hapus semua siswa yang terhubung ke jurusan ID 3
```

Penjelasan: semua siswa jurusan 3 terhapus, bukan hanya satu.

d. Dampak foreign key

Aksi	Efek
Hapus **siswa**	Nilai milik siswa ikut terhapus (`CASCADE`)
Hapus **jurusan**	`id_jurusan` siswa terkait menjadi `NULL` (`SET NULL`)
Ubah `id_jurusan` di tabel `jurusan`	Ikut berubah di `siswa` (`ON UPDATE CASCADE`)

Coba pada data latihan, lalu amati hasilnya dengan `SELECT`.

e. Contoh `DELETE` berbahaya (jangan dijalankan)

```
-- Tanpa WHERE: seluruh isi tabel terhapus
-- DELETE FROM siswa;
```

5.4 Transaksi sederhana untuk percobaan aman (opsional)

Pada tabel InnoDB, kamu bisa mencoba `UPDATE`/`DELETE` lalu membatalkannya:

```
START TRANSACTION;

DELETE FROM nilai WHERE id_siswa = 1; -- hapus sementara semua nilai siswa ID 1
SELECT * FROM nilai;                  -- periksa perubahan sebelum transaksi diputuskan

ROLLBACK;                             -- batalkan perubahan sejak START TRANSACTION
SELECT * FROM nilai;                  -- pastikan nilai siswa ID 1 kembali
```

Jalankan dalam **satu sesi koneksi yang sama**. `START TRANSACTION` memulai transaksi, `ROLLBACK` membatalkan perubahan yang belum di-`COMMIT`. Perhatikan bahwa phpMyAdmin pada pengaturan tertentu menjalankan *autocommit*; terminal lebih mudah untuk percobaan ini.

---

6. Rangkuman

Query	Fungsi	`WHERE`
`INSERT INTO`	Menambah baris	Tidak diperlukan
`INSERT ... SELECT`	Menambah baris dari hasil query	Opsional di `SELECT`
`UPDATE`	Mengubah baris	**Sangat disarankan**
`DELETE FROM`	Menghapus baris	**Sangat disarankan**

Aturan emas: `SELECT` dulu, baru `UPDATE` / `DELETE`.

---

7. Latihan

Gunakan `db_sekolah` dengan data awal.

1. Tambahkan siswa baru: **Hana**, perempuan, lahir `2008-12-01`, jurusan TKJ (ID 3).
2. Tambahkan dua mapel sekaligus: **Algoritma** dan **Pemrograman Dasar**.
3. Tambahkan nilai Hana untuk mapel Basis Data sebesar 89 (semester 1). *Petunjuk: cek dulu ID siswa dan mapel dengan `SELECT`.*
4. Ubah email Siti menjadi `siti.nur@example.com`.
5. Naikkan nilai Basis Data semester 1 sebesar 2 poin untuk semua siswa.
6. Hapus nilai milik siswa ber-ID 3 pada mapel ber-ID 2.
7. Jelaskan apa yang terjadi pada tabel `nilai` jika seorang siswa dihapus. Buktikan dengan percobaan.

Pertanyaan diskusi: apa risikonya menjalankan `UPDATE` tanpa `WHERE`, dan bagaimana cara mencegahnya?

---

8. Kesalahan Umum

Masalah	Penyebab	Solusi
`Duplicate entry`	Email/nama unik sudah ada	Periksa data, ganti nilai
`Cannot add or update a child row`	Nilai FK tidak ada di tabel induk	Isi tabel induk lebih dulu
`Column count doesn't match`	Jumlah kolom dan nilai berbeda	Samakan jumlah dan urutan
Semua baris berubah/hilang	Lupa `WHERE`	Selalu uji dengan `SELECT`
`Incorrect ... value` pada tanggal	Format salah	Gunakan `YYYY-MM-DD`